

DESIGN SPECIFICATION DOCUMENT

HCI with IoT & Embedded Systems

Project Title: SENTINEL EYE - AI VIRTUAL FENCE SYSTEM

Team Name: *[YOUR TEAM NAME]*

Date: *[SUBMISSION DATE]*

SUBMISSION TIMELINE

- **Draft (Sections 1-5):** Complete BEFORE Session 2 for discussion
- **Final Document (All Sections):** Submit by Session 4 for graded presentation
- **Instructions:** Replace all *[gray italic text in brackets]* with your actual content

Document Sections

1. Project Overview
2. Related Work & Inspiration
3. User Research (Personas & Scenarios)
4. System Architecture
5. Hardware Design
6. Software Design
7. AI/ML Component
8. UI/UX Design
9. Testing Plan
10. Risk Assessment
11. Success Metrics
12. Project Timeline
13. Team Communication Plan
14. Appendix

Section 1: Project Overview

Project Title	SENTINEL EYE: Hệ thống Giám sát An ninh Hàng rào ảo Thông minh (AI Virtual Fence)
Team Name	[Your creative team name]
Team Members	1. [Full Name] — [Role: e.g., Team Lead / Hardware] — [Student ID] — [Email] 2. [Full Name] — [Role: e.g., ML Developer] — [Student ID] — [Email] 3. [Full Name] — [Role: e.g., UI/Dashboard] — [Student ID] — [Email] 4. [Full Name] — [Role: e.g., Testing/Docs] — [Student ID] — [Email]
Problem Statement	Trong an ninh quân sự và dân sự, việc giám sát các khu vực nhạy cảm (kho bãi, biên giới, vùng nguy hiểm) thường phụ thuộc vào hàng rào vật lý hoặc con người túc trực 24/7. Các cảm biến chuyển động truyền thống (PIR) thường gây báo động giả do động vật hoặc lá cây. Trong khi đó, hệ thống Camera AI chuyên nghiệp lại quá đắt đỏ và phức tạp để triển khai nhanh tại thực địa. Người vận hành thiếu một công cụ trực quan để thiết lập vùng bảo vệ linh hoạt theo thời gian thực mà không cần lắp đặt lại phần cứng.
Proposed Solution	Chúng tôi xây dựng Sentinel Eye , một hệ thống giám sát cơ động sử dụng ESP32-CAM làm thiết bị thu thập hình ảnh không dây giá rẻ. Sức mạnh xử lý nằm ở Server (Laptop/PC) chạy các thuật toán Deep Learning (YOLO) để phát hiện con người. Điểm đặc biệt về HCI là người dùng có thể vẽ trực tiếp hàng rào ảo (Polygon) lên video stream trên Web App. Khi đối tượng xâm nhập vào vùng vẽ, hệ thống sẽ cảnh báo tức thì bằng âm thanh và hình ảnh.
Project Scope	In Scope (Những gì sẽ làm): • Phát hiện người (Human Detection) sử dụng mô hình YOLO. • Giao diện Web cho phép vẽ đa giác (Polygon) tùy ý lên Video Stream. • Thuật toán kiểm tra xâm nhập: Point-in-Polygon (Ray Casting). • Cảnh báo thời gian thực: Web Visual Alert & ESP32 Buzzer. • Lưu trữ lịch sử xâm nhập (Log kèm ảnh chụp). Out of Scope (Những gì không làm): • Nhận diện danh tính cụ thể (Face Recognition - do hạn chế độ phân giải camera). • Hoạt động trong bóng tối hoàn toàn (Night Vision). • Chống nước/bụi chuẩn công nghiệp.

Section 2: Related Work & Inspiration

Research at least 3 existing products or projects similar to yours. This helps you understand the competitive landscape and identify opportunities for differentiation.

Product/Project	Key Features	Limitations	Your Differentiation
Ring / Nest Cam (Commercial)	Camera an ninh gia đình, có khoanh vùng	Giá thành cao (\$100+), phụ thuộc vào Cloud,	**Chi phí cực thấp (<\$10)**, xử lý tại Local

Product/Project	Key Features	Limitations	Your Differentiation
	chuyển động (Zone Detection).	độ trễ cao, khó tùy biến.	Server (Bảo mật), tùy biến được logic cảnh báo.
Cảm biến PIR (Truyền thống)	Phát hiện chuyển động bằng hồng ngoại, giá rẻ.	Báo động giả rất nhiều (gió, mèo, lá cây), không biết vị trí chính xác.	Sử dụng Computer Vision để lọc chính xác là "Người", loại bỏ nhiễu môi trường.
Các đồ án ESP32-CAM cũ	Thường chỉ stream video đơn thuần hoặc chụp ảnh lưu thẻ nhớ.	Không có xử lý AI thời gian thực, UX kém (chỉ là trang web tĩnh).	Tập trung mạnh vào UX (Vẽ tương tác) và AI (YOLO) , biến ESP32 thành một thiết bị IoT thông minh thực thụ.

Key Insights from Research

- Các giải pháp hiện tại hoặc quá đắt (Commercial) hoặc quá "ngu" (Sensor vật lý).
- Người dùng cần sự linh hoạt: Sáng vẽ hàng rào ở cổng, tối vẽ hàng rào quanh xe máy. Phần mềm phải đáp ứng được điều này (Interactive Canvas).

Section 3: User Research

3.1 Target Users

Primary User Group	Nhân viên an ninh/trực ban (trong ngữ cảnh quân sự/công ty) hoặc Chủ nhà (ngữ cảnh dân sự).
Context of Use	Ngồi tại phòng điều khiển, quan sát qua màn hình Laptop/Tablet.
User Needs	Cần thiết lập vùng cấm nhanh chóng bằng chuột/cảm ứng. Cần cảnh báo rõ ràng để không phải dán mắt vào màn hình liên tục.

3.2 User Personas

Create at least 2 detailed personas representing your target users.

Persona 1 : Trung sĩ An (The Watchman)

Background	30 tuổi, trực ban kho vũ khí đơn vị.
Goals	Đảm bảo không ai lại gần khu vực cửa kho sau 22:00.
Pain Points	Hay buồn ngủ khi phải nhìn màn hình camera tĩnh quá lâu. Camera cũ cứ báo động khi có chó chạy qua khiến anh mất cảnh giác.
Tech Comfort Level	Trung bình. Cần giao diện dễ hiểu, ít nút bấm.

Persona 2: Dev Tuấn (The Tech Enthusiast)

Background	22 tuổi, sinh viên thuê trọ, có nhiều đồ công nghệ đắt tiền.
Goals	Giám sát phòng trọ khi đi học. Muốn hệ thống tự chụp ảnh gửi về Telegram khi có người lạ mở cửa.
Pain Points	Không muốn khoan tường lắp đặt thiết bị phức tạp. Muốn tận dụng laptop cũ làm server.

3.3 User Scenarios

Describe at least 2 realistic scenarios showing how your personas would use the system.

Scenario 1 : Thiết lập hàng rào (Setup Phase)

Context	Trung sĩ An bắt đầu ca trực tối. Anh đặt ESP32-CAM hướng ra sân.
---------	--

Actions	Trên Web App, anh dùng chuột chấm 4 điểm tạo thành hình tứ giác bao quanh chiếc xe Jeep chỉ huy. Anh nhấn nút "Armed" (Kích hoạt).
Outcome	Hệ thống xác nhận vùng bảo vệ đã được thiết lập.

Scenario 2: Phát hiện xâm nhập (Operation Phase)

Context	Một lính mới tò mò đi lại gần chiếc xe Jeep.
Actions	Camera thu hình -> Server phát hiện "Person" -> Tọa độ chân người lính nằm trong vùng tứ giác đỏ.
Outcome	Còi trên ESP32 hú vang "Bíp Bíp", màn hình Laptop nháy đỏ và chụp lại khoảnh khắc đó lưu vào lịch sử.

Section 4: System Architecture

4.1 Six-Layer HCI Architecture

Map your system components to the 6-layer HCI ecosystem framework:

Layer	Your Components	Description & Purpose
6. Feedback	Web Dashboard (Visual) + Buzzer (Audio)	Cảnh báo trực quan (khung đỏ) và âm thanh tại chỗ để xua đuổi kẻ gian.
5. Presentation	Web App (ReactJS/HTML5 Canvas)	Hiển thị Video Stream, cung cấp công cụ vẽ (Draw Tool) để người dùng tương tác.
4. Processing	Laptop Server (Python/Flask + YOLOv8)	"Bộ não" của hệ thống. Nhận ảnh, chạy AI detect người, tính toán hình học (Point-in-Polygon).
3. Communication	WiFi (HTTP / WebSocket)	Truyền video từ ESP32 -> Server. Truyền lệnh Bật/Tắt còi từ Server -> ESP32.
2. Embedded	ESP32-CAM	Thiết bị biên (Edge Device). Thu thập hình ảnh và điều khiển phần cứng cơ bản.
1. Physical	Camera OV2640, Active Buzzer	Cảm biến hình ảnh đầu vào và thiết bị phát âm thanh đầu ra.

4.2 System Block Diagram

<i>[INSERT SYSTEM BLOCK DIAGRAM HERE]</i> Show data flow: Tools: draw.io, Lucidchart, Figma, or hand-drawn scan Label all components, connections, and data types clearly

4.3 Data Flow Description

[Describe in words how data flows through your system from sensor input to user feedback. Include data formats, transformation steps, and timing considerations.]

- **Input:** ESP32-CAM quay video (MJPEG stream) gửi qua WiFi đến Server Python.
- **Processing (AI):** Server nhận từng frame hình, đưa qua model YOLOv8 để lấy bounding box (x, y, w, h) của con người.

- **Processing (Logic):** Server lấy tọa độ trung điểm đáy của box (chân người) so sánh với tọa độ các điểm của đa giác (Polygon) mà người dùng đã vẽ trên Web.
- **Decision:** Nếu điểm chân người nằm TRONG đa giác -> Trạng thái = **INTRUSION**.
- **Output:** Server gửi lệnh HTTP GET đến ESP32 để bật Buzzer. Web UI hiển thị cảnh báo đỏ.

Section 5: Hardware Design

5.1 Bill of Materials (BOM)

Component	Model/Part #	Qty	Unit Price	Subtotal	Source/Link
Main Board	ESP32-CAM (AI-Thinker)	1	140,000	140,000	
Programmer	Đế nạp ESP32-CAM MB (USB)	1	60,000	60,000	
Camera	OV2640 (Kèm theo board)	1	0	0	
Feedback	Còi chirp Active Buzzer 5V	1	5,000	5,000	[...]
Power	Cáp Micro USB	1	10,000	10,000	
Wiring	Dây Jumper Cái-Cái	5	5,000	5,000	[...]
TOTAL COST				~ 220,000 VND	

5.2 Wiring Diagram

[INSERT WIRING/CIRCUIT DIAGRAM HERE]

- **Webcam:** Kết nối qua cổng USB 2.0/3.0 của Raspberry Pi.
- **Buzzer (Còi):** Chân dương (+) nối vào GPIO 17. Chân âm (-) nối vào GND.
- **LED (Đèn):** Chân dương (dài) nối vào GPIO 27 qua điện trở 220Ω. Chân âm (ngắn) nối vào GND.
- **Button (Nút):** Một chân nối vào GPIO 22. Chân chéo đối diện nối vào GND (Sử dụng code Python để kích hoạt điện trở kéo lên - Internal Pull-up Resistor).

Tools: Fritzing, TinkerCAD Circuits, or hand-drawn scan
Label all pins and color-code wires (red=VCC, black=GND, etc.)

5.3 Pin Connection Table

ESP32 Pin	Component Pin	Protocol/Notes
5V / VCC	Nguồn USB	Cấp nguồn
GND	GND (Buzzer)	Nối đất
GPIO 12 (hoặc 14)	Chân Dương (+) Buzzer	Điều khiển còi (High = Kêu)
Camera Socket	OV2640 Camera	Cắm sẵn trên board

Section 6: Software Design

6.1 Technology Stack

Layer	Technology Choice	Justification
Embedded	C++ (Arduino IDE) - Code cực nhẹ, chỉ để stream video và bật tắt GPIO.	<i>[Why this choice? e.g., familiarity, library support]</i>
Backend & AI	Python (Flask/FastAPI) - Dễ dàng tích hợp thư viện AI.	<i>[Why? e.g., real-time DB, easy auth, free tier]</i>
Frontend	HTML5, CSS, JavaScript (Fabric.js hoặc Canvas API) - Để vẽ đa giác lên video.	<i>[Why? e.g., rapid prototyping, visualization features]</i>
AI Model	YOLOv8 Nano (Ultralytics) - Phiên bản nhẹ nhất, chạy mượt trên CPU Laptop.	<i>[Why? e.g., real-time needs, power constraints]</i>

6.2 Key Algorithms & Logic

Describe the main algorithms and logic that power your system:

Data Processing Algorithm

[Describe how raw sensor data is processed: filtering, normalization, feature extraction. Include pseudocode or flowchart if helpful.]

1. Ray Casting Algorithm (Point in Polygon): Thuật toán hình học cốt lõi để kiểm tra xem một điểm (tọa độ người) có nằm trong hình đa giác (vùng cấm) hay không.

- *Input:* Tọa độ điểm P(x,y), Danh sách đỉnh đa giác.
- *Logic:* Kẻ một tia từ P ra vô cực, đếm số lần cắt cạnh đa giác. Số lẻ = Trong, Số chẵn = Ngoài.

Decision Logic

[Describe how your system makes decisions: thresholds, rules, ML inference results. When does it trigger alerts/actions?]

2. Alert Debounce Logic: Để tránh còi kêu ngắt quãng khó chịu, hệ thống sẽ duy trì trạng thái báo động trong ít nhất 3 giây nếu phát hiện xâm nhập, ngay cả khi AI bị mất dấu trong tích tắc.

6.3 API Design (if applicable)

Define key API endpoints if your system includes backend communication:

Method	Endpoint	Description	Request/Response
POST	<i>[/api/sensor-data]</i>	<i>[Submit new sensor reading]</i>	<i>[{br, spo2, timestamp}]</i>
GET	<i>[/api/history]</i>	<i>[Get historical data]</i>	<i>[Array of readings]</i>
...	<i>[Add more endpoints...]</i>	<i>[...]</i>	<i>[...]</i>

Section 7: AI/ML Component

ML Task Type	<i>[Classification / Regression / Anomaly Detection / Time Series Forecasting]</i>
Model Architecture	<i>[e.g., 2-layer CNN, Random Forest, LSTM, Decision Tree, etc.]</i>
Input Features	<i>[List all input features with dimensions:]</i> • Feature 1: <i>[e.g., heart_rate - float, 1 value]</i> • Feature 2: <i>[e.g., accel_xyz - float array, 3 values]</i> • Window size: <i>[e.g., 100 samples at 50Hz = 2 seconds]</i>
Output	<i>[e.g., 3 classes: Normal, Warning, Critical / or continuous value 0-100]</i>
Training Data Source	<i>[Self-collected / Public dataset (name & link) / Synthetic — specify total size]</i>
Labeling Strategy	<i>[How will you label data? Manual annotation / threshold-based / existing labels]</i>
Training Platform	<i>[Edge Impulse Studio / Google Colab / Local Python / Jupyter Notebook]</i>
Deployment Target	<i>[On-device (ESP32) / Cloud inference / Hybrid (edge + cloud)]</i>
Model Size Constraint	<i>[e.g., <100KB for ESP32 flash, <50KB RAM for inference]</i>
Target Accuracy	<i>[e.g., >85% accuracy, <10% false positive rate, >90% recall]</i>
Inference Time Target	<i>[e.g., <100ms per prediction for real-time response]</i>

7.2 Data Collection Plan

[Describe your plan for collecting training data: How many samples? How will you collect them? How will you ensure data quality and diversity?]

7.3 Model Validation Strategies

[How will you validate your model? Train/test split ratio? Cross-validation? Confusion matrix analysis? Real-world testing?]

Section 8: UI/UX Design

8.1 Dashboard Wireframes

Include wireframes or mockups for at least 3 screens. Use Figma, Adobe XD, Balsamiq, or hand-drawn sketches.

Screen 1: Main Dashboard

[INSERT WIREFRAME IMAGE HERE]

Description: *[What does this screen show? Key elements and their purpose]*

Screen 2: Detail/History View

[INSERT WIREFRAME IMAGE HERE]

Description: *[What does this screen show?]*

Screen 3: Settings/Configuration

[INSERT WIREFRAME IMAGE HERE]

Description: *[What can users configure?]*

8.2 Feedback Mechanisms

Describe how your system provides feedback to users through multiple modalities:

Feedback Type	Implementation	Trigger Condition	User Benefit
Visual	<i>[RGB LED colors, screen alerts]</i>	<i>[e.g., Red when HR > 120 BPM]</i>	<i>[Immediate visual awareness]</i>
Audio	<i>[Buzzer tones, speaker alerts]</i>	<i>[e.g., Beep on anomaly detected]</i>	<i>[Alert when not looking at screen]</i>
Haptic	<i>[Vibration motor patterns]</i>	<i>[e.g., Vibrate for critical alerts]</i>	<i>[Discreet notification]</i>
Notification	<i>[Push notification, email, SMS]</i>	<i>[e.g., Daily summary, emergencies]</i>	<i>[Remote awareness]</i>
Dashboard	<i>[Charts, gauges, status indicators]</i>	<i>[Real-time data display]</i>	<i>[Detailed information access]</i>

Section 9: Testing Plan

Test Type	What to Test	Testing Method	Success Criteria
Unit Testing	<i>[Individual sensor functions, data processing functions]</i>	<i>[Manual testing, Serial Monitor output verification]</i>	<i>[All functions return expected values]</i>
Integration Testing	<i>[Sensor → MCU → Cloud → Dashboard data flow]</i>	<i>[End-to-end data flow verification]</i>	<i>[Data arrives at dashboard within 2 seconds]</i>
Hardware Testing	<i>[Sensor accuracy, power consumption, physical durability]</i>	<i>[Compare with reference device, measure with multimeter]</i>	<i>[Within $\pm 5\%$ of reference]</i>
ML Model Testing	<i>[Model accuracy, inference time, edge cases]</i>	<i>[Test dataset, confusion matrix, timing measurements]</i>	<i>[>85% accuracy, <100ms inference]</i>
User Testing	<i>[Usability, task completion, satisfaction]</i>	<i>[SUS questionnaire, Think-aloud protocol, observation]</i>	<i>[SUS score >70, task completion >90%]</i>
Stress Testing	<i>[Long-running stability, memory leaks, error recovery]</i>	<i>[Run system for 24+ hours continuously]</i>	<i>[No crashes, stable memory usage]</i>

Section 10: Risk Assessment

Risk Description	Likelihood	Impact	Priority	Mitigation Strategy
<i>[Hardware delivery delayed]</i>	Medium	High	High	<i>[Order early, have backup vendor, use simulation initially]</i>
<i>[Sensor readings inaccurate]</i>	Medium	Medium	Medium	<i>[Implement calibration, use filtering algorithms, validate against reference]</i>
<i>[ML model underperforms]</i>	Medium	High	High	<i>[Collect more training data, try simpler model, use rule-based fallback]</i>
<i>[Team member unavailable]</i>	Low	Medium	Medium	<i>[Cross-train team members, document everything, regular check-ins]</i>
<i>[WiFi connectivity issues]</i>	Medium	Low	Low	<i>[Implement local data buffering, offline mode, connection retry logic]</i>
<i>[Add more risks...]</i>	<i>[L/M/H]</i>	<i>[L/M/H]</i>	<i>[L/M/H]</i>	<i>[Mitigation plan...]</i>

Section 11: Success Metrics

Category	Metric	Target Value
Technical - Hardware	Sensor accuracy	[e.g., $\pm 5\%$ of reference device]
	System response time	[e.g., $< 500\text{ms}$ end-to-end]
	Battery life (if applicable)	[e.g., > 8 hours continuous use]
	Uptime/reliability	[e.g., $> 95\%$ uptime over 24 hours]
Technical - Software	Data transmission success rate	[e.g., $> 99\%$ packets delivered]
	Dashboard refresh rate	[e.g., updates every 1 second]
	Error handling coverage	[e.g., graceful handling of all known error cases]
AI/ML	Classification accuracy	[e.g., $> 85\%$ on test set]
	Inference time	[e.g., $< 100\text{ms}$ on ESP32]
	False positive rate	[e.g., $< 10\%$]
	False negative rate	[e.g., $< 5\%$ for critical events]
User Experience	SUS (System Usability Scale) Score	[e.g., $> 70/100$ (above average)]
	Task completion rate	[e.g., $> 90\%$ of users complete all tasks]
	Time to complete key tasks	[e.g., < 30 seconds for primary action]
	User satisfaction rating	[e.g., 4+ out of 5 stars]

Section 12: Project Timeline

Session	Milestone	Tasks	Owner(s)
1-2	Planning	[Team formation, topic selection, design spec draft, hardware order]	[All team members]
3-4	Hardware Setup	[ESP32 setup, sensor integration, design spec finalization, presentation]	[Hardware Lead: Name]
5-6	Data & AI	[Data collection, signal processing, model training, ESP32 deployment]	[ML Lead: Name]
7-8	UI/Dashboard	[Dashboard development, visualization, feedback mechanisms]	[UI Lead: Name]
9-10	Integration & Test	[System integration, end-to-end testing, user testing, iteration]	[All team members]
11	FINAL DEMO	Final presentation, live demo, report submission, video	All team members

Section 13: Team Communication Plan

Primary Communication Tool	<i>[e.g., Discord server / Slack workspace / Zalo group — include invite link]</i>
Meeting Schedule	<i>[e.g., Every Thursday 7-8 PM online + additional meetings as needed]</i>
Code Repository	<i>[GitHub repository URL — create this early! e.g., github.com/team/project]</i>
File Sharing	<i>[Google Drive folder URL for documents, images, datasets]</i>
Task Tracking	<i>[Trello board / Notion workspace / GitHub Projects — include link]</i>
Decision Log	<i>[Where will you document key decisions? e.g., shared document, Notion page]</i>

Team Roles & Responsibilities

Team Member	Primary Role	Responsibilities
<i>[Name 1]</i>	<i>[Team Lead / Hardware]</i>	<i>[Coordinate team, manage timeline, hardware design & assembly]</i>
<i>[Name 2]</i>	<i>[ML Developer]</i>	<i>[Data collection, model training, deployment, accuracy optimization]</i>
<i>[Name 3]</i>	<i>[UI/Dashboard]</i>	<i>[Dashboard design & development, user interface, visualization]</i>
<i>[Name 4]</i>	<i>[Testing/Docs]</i>	<i>[Testing, documentation, user studies, presentation materials]</i>

Appendix

Use this section for supplementary materials:

A. Additional Diagrams

[Insert any additional technical diagrams, flowcharts, state machines, etc.]

B. Code Snippets

[Include key code snippets for important algorithms or functions — not full source code]

C. User Survey Templates

[Include SUS questionnaire, custom survey questions, interview scripts]

D. References

[List all references: academic papers, tutorials, documentation, datasets used]

E. Meeting Notes

[Summary of key team meetings and decisions made]